영남대학교 Agentic AI 강의 과정

Agentic AI 개념, AI 3대장 비교(1주차 - 2일차 이론)

Claude를 활용한 바이브 코딩으로 웹사이트 개선 & 네이버클라우드의 웹서버로 배포

일정: 2026. 11. 01(일) 13:00 ~ 18:00
과정: Agentic AI 실전 개발 & 클라우드 인프라 연동 (총 8일 / 40시간)
강사: 신인철 (클라우드스퀘어)

🕐 2일차 — Agentic AI와 Claude 를 활용한 바이브 코딩으로 웹 사이트 개선

2026. 11. 01(일) 13:00 ~ 18:00 시간표

이론 3시간 | 실습 1.5시간 | 정리 30분 | 50분 수업 + 10분 휴식 | 토 · 일 13:00 ~ 18:00 · 총 8일 / 40시간

시간	구분	내용
13:00 ~ 13:50	이론 1	1일차 회고 · LLM과 Agentic AI 개념
13:50 ~ 14:00	휴식	☕ 10분 휴식
14:00 ~ 14:50	이론 2	AI 3대장 비교 · 바이브 코딩과 좋은 프롬프트
14:50 ~ 15:00	휴식	☕ 10분 휴식
15:00 ~ 15:50	이론 3	웹 기초(HTML · CSS · JS) · 배포 원리(HTTP · SSH)
15:50 ~ 16:00	휴식	☕ 10분 휴식
16:00 ~ 16:50	실습 ①	웹사이트 기획 → Claude로 첫 버전 → 웹 사이트 개선 (STEP 0~3)
16:50 ~ 17:00	휴식	☕ 10분 휴식
17:00 ~ 17:30	실습 ②	서버 접속 → 배포 → 브라우저 확인 (STEP 4~6)
17:30 ~ 18:00	정리	결과 제출 · 회고 · Q&A

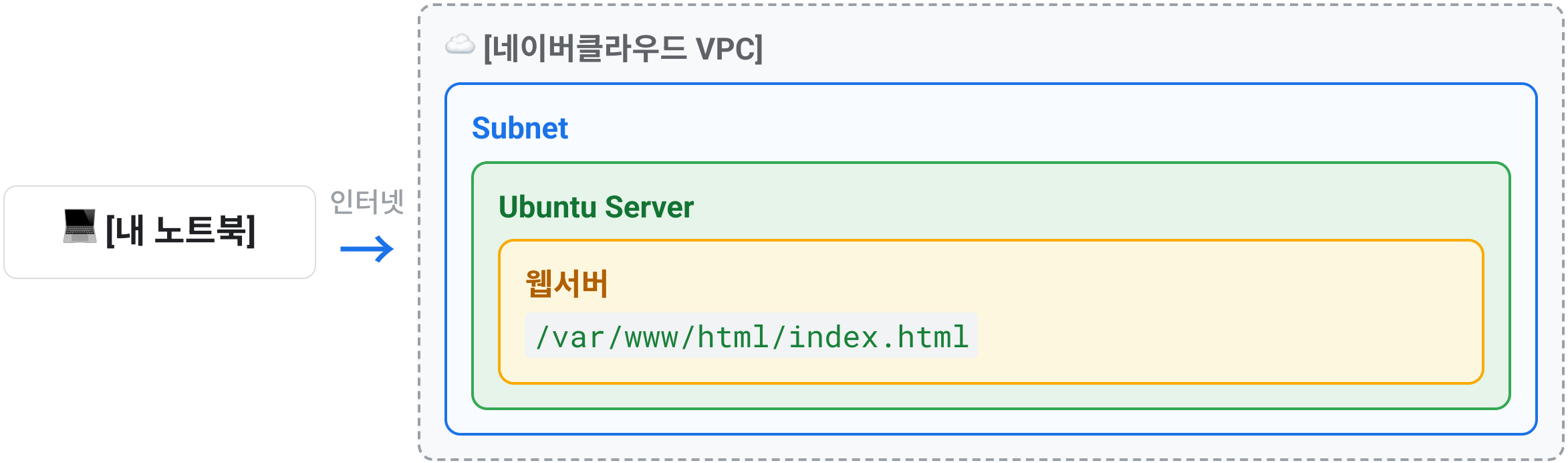
1일차 복습

어제 우리는 이것을 해냈습니다.

1. 네이버클라우드에 **VPC / Subnet** 생성
2. **Ubuntu Linux Server** 생성
3. 웹서버 설치 및 실행
4. 예제 `index.html`을 올려 **브라우저에서 내 서버 접속**

> 오늘은 이 웹서버 위에 **AI와 함께 만든 나만의 웹사이트**를 올립니다.

어제의 구조 다시 보기



- 브라우저 주소창에 `http://<공인 IP>` 입력 → `index.html`이 보임
- 오늘은 이 **`index.html`을 교체**하는 것이 최종 목표

오늘의 흐름

구분	내용
이론 1	1일차 복습
이론 2	Agentic AI란 무엇인가
이론 3	AI 3대장 비교: Claude · OpenAI · Google 과 좋은 프롬프트
실습 1	Claude 를 활용한 바이브 코딩으로 웹 사이트 개선
실습 2	웹서버 배포 방법 (원격 접속)
실습 3	Claude로 웹사이트 개선 → 네이버클라우드 웹서버 배포

🎯 오늘의 학습 목표

오늘 수업이 끝나면 여러분은 다음을 할 수 있습니다.

#	할 수 있게 되는 것	관련 Part
1	**Agentic AI**가 챗봇과 무엇이 다른지 내 말로 설명할 수 있다	Part 1
2	Claude · ChatGPT · Gemini를 **용도에 맞게 고르는 기준**을 말할 수 있다	Part 2
3	**좋은 프롬프트의 4요소**로 원하는 웹사이트를 요청할 수 있다	Part 3
4	AI가 만든 코드를 **질문하며 이해**하고, 구체적으로 수정 요청할 수 있다	Part 3
5	내가 만든 `index.html`을 **원격 접속으로 서버에 배포**할 수 있다	Part 4

💡 **오늘의 키워드**: 에이전트 · 바이브 코딩 · 프롬프트 · 반복 개선 · 원격 접속 · 배포

🔁 1일차 핵심 용어 다시 짚기

용어	한 줄 설명	오늘 쓰는 곳
공인 IP	인터넷에서 내 서버를 찾는 주소	브라우저 접속 · 원격 접속 주소
root 비밀번호	서버 관리자 계정의 비밀번호 (인증키 `.pem` 으로 확인)	VS Code · MobaXterm · 터미널 로그인
ACG	서버의 방화벽, 허용한 포트만 통과	**22번**(SSH) · **80번**(HTTP)
웹서버 (Nginx)	브라우저 요청에 웹 파일을 보내주는 프로그램	배포한 파일을 보여줌
문서 루트	웹 파일을 두는 폴더 `/var/www/html`	`index.html` 교체 위치

ℹ️ **확인 질문**: 브라우저에 `http://<공인 IP>` 를 입력했을 때 페이지가 보이려면, ACG에서 **몇 번 포트**가 열려 있어야 할까요? → **80번**

Part 1

Agentic AI란 무엇인가

먼저, 생성형 AI와 LLM 이해하기

LLM (Large Language Model, 대규모 언어 모델): 엄청난 양의 글을 학습해서 **"다음에 올 가장 자연스러운 말"**을 이어서 만들어 내는 AI

① 입력
프롬프트
"자기소개 홈페이지를 만들어줘"

→

② 이해
문맥 파악
요청 · 이전 대화 · 첨부 자료를 함께 읽음

→

③ 생성
한 조각씩 이어 쓰기
단어(토큰)를 하나씩 예측하며 문장 · 코드 생성

→

④ 출력
응답
설명 · HTML 코드 · 표 · 요약

- 글뿐 아니라 **코드**도 "언어"이기 때문에 LLM은 프로그램도 작성할 수 있습니다.
- Claude · ChatGPT · Gemini 모두 **LLM을 두뇌로 쓰는 서비스**입니다.

LLM이 잘하는 것과 조심할 것

잘하는 것 ✅	조심할 것 ⚠️
글 · 코드를 **빠르게 초안** 작성	**그럴듯한 오답**(환각, Hallucination)을 자신 있게 말하기도 함
긴 문서 **요약 · 번역 · 정리**	학습 이후의 **최신 정보**는 모를 수 있음
예시를 보고 **형식 맞추기**	대화가 너무 길면 앞부분을 **놓칠 수 있음**
설명을 **쉬운 말로 바꾸기**	내 서버 · 내 파일 상황은 **직접 보지 못함** (챗봇 방식일 때)

❗ **결론**: AI의 답은 **"검토할 초안"** 입니다. 코드는 **실행해서 확인**하고, 사실은 **공식 자료로 확인**하는 습관이 필요합니다.

컨텍스트 — AI의 "작업 기억"

- **컨텍스트(Context)**: AI가 답을 만들 때 참고하는 **지금까지의 대화 + 첨부 자료 전체**
- 한 대화 안에서 앞서 말한 조건(예: "파란색 계열")을 기억하는 이유가 바로 컨텍스트

상황	일어나는 일	대처 방법
같은 대화에서 계속 수정	이전 요청을 기억하며 수정 ✅	그대로 이어서 요청
대화가 너무 길어짐	앞의 조건을 놓치거나 느려짐	**최종 코드만 새 채팅에** 붙여넣고 이어가기
새 채팅을 시작	이전 대화를 모름	필요한 조건 · 코드를 **다시 알려주기**

> 💡 실습 3-2 "수정이 꼬였을 때"의 **"새 채팅에 붙여넣기"** 가 바로 이 원리입니다.

AI 활용 방식의 변화

단계	방식	사람의 역할
챗봇	질문하면 답변	매번 묻고, 결과를 직접 적용
코파일럿	옆에서 제안	제안을 보고 수락/거절
에이전트	**목표를 주면 스스로 계획·실행**	**목표 설정과 결과 검토**

> AI가 "대답하는 도구"에서 "일을 해내는 동료"로 바뀌고 있습니다.

비유로 보는 챗봇 · 코파일럿 · 에이전트

1단계 챗봇

🗺️ 길을 물어보기

"서울역 어떻게 가요?" → 설명을 듣고 **내가 직접** 운전

코딩에서는

코드를 받아 **사람이** 복사 · 붙여넣기 · 실행

2단계 코파일럿

🧭 내비게이션

운전 중에 옆에서 "다음에서 우회전" 안내 → 판단은 **내가**

코딩에서는

코드를 치는 동안 다음 줄을 **제안** → 수락 / 거절

3단계 에이전트

🚗 자율주행

"서울역까지" 목표만 주면 **스스로** 경로를 정하고 운전 · 상황 대응

코딩에서는

파일 수정 · 명령 실행 · 오류 수정까지 **스스로** → 사람은 목표 · 검토

ℹ️ **오늘은 1단계(챗봇) 방식**으로 Claude와 협업합니다. 2주차에는 3단계(에이전트)를 직접 체험합니다.

비유로 보는 챗봇 · 코파일럿 · 에이전트 (확대 보기)

1단계 챗봇

🗺️ 길을 물어보기

"서울역 어떻게 가요?" → 설명을 듣고
내가 직접
운전

코딩에서는
코드를 받아
사람이
복사 · 붙여넣기 · 실행

2단계 코파일럿

🧭 내비게이션

운전 중에 옆에서 "다음에서 우회전" 안내 → 판단은
내가

코딩에서는
코드를 치는 동안 다음 줄을
제안
→ 수락 / 거절

3단계 에이전트

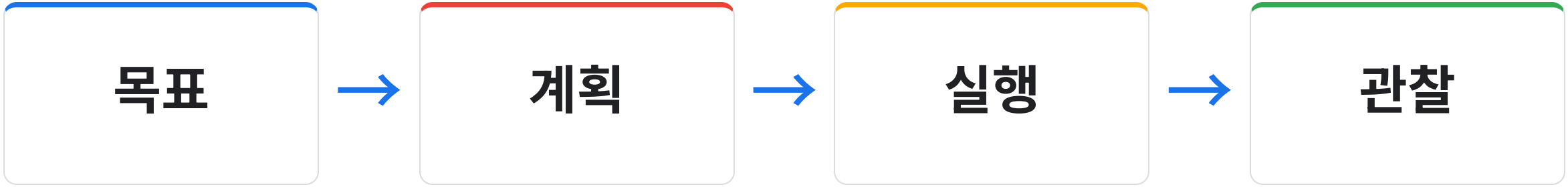
🚗 자율주행

"서울역까지" 목표만 주면
스스로
경로를 정하고 운전 · 상황 대응

코딩에서는
파일 수정 · 명령 실행 · 오류 수정까지
스스로
→ 사람은 목표 · 검토

Agentic AI의 정의

목표를 받으면 스스로 계획을 세우고, 도구를 사용해 실행하고,
결과를 확인하며 목표를 달성할 때까지 반복하는 AI



↻ 관찰 ——▶ **수정** ——▶ 다시 계획 (목표를 달성할 때까지 반복)

에이전트 루프 따라가 보기

목표: "내 웹사이트에 다크모드 버튼을 추가하고 서버에 반영해줘"

단계	에이전트가 하는 일 (예시)
계획	① `index.html` 구조 확인 ② 버튼 · CSS · JS 추가 ③ 브라우저 확인 ④ 서버 반영
실행	`index.html` 을 열어 버튼과 다크모드 스타일 · 스크립트를 추가
관찰	페이지를 열어 보니 **버튼은 보이지만 글자색이 안 바뀜**
수정	원인 분석 → CSS 선택자 수정 → 다시 실행
관찰	정상 동작 확인 ✅
완료 보고	"다크모드 버튼 추가 완료, 변경 파일: `index.html` "

> 💡 핵심은 **"관찰 → 수정"의 반복**입니다. 사람이 매번 알려주지 않아도 결과를 보고 스스로 고칩니다.

에이전트의 4가지 구성 요소

구성 요소	역할	비유
LLM	판단하고 생각하는 두뇌	사람의 두뇌
도구 (Tools)	파일 작성, 명령 실행, 검색 등	손과 발
컨텍스트 / 메모리	지금까지의 작업 내용 기억	작업 노트
계획 (Planning)	큰 목표를 작은 단계로 분해	할 일 목록

도구(Tools)가 에이전트를 "일하게" 만든다

- LLM은 원래 **글만 만들 수 있는 두뇌**
- **도구**를 연결하면 실제로 **행동**할 수 있음

도구 예시	할 수 있는 일
파일 읽기 · 쓰기	코드 파일 수정
터미널 명령 실행	설치 · 실행 · 배포
웹 검색	최신 정보 확인
외부 서비스 연결	메일 · 일정 · 문서 처리



💡

MCP (Model Context Protocol): AI와 여러 도구 · 서비스를 연결하는 **공개 표준 규격**. "AI용 USB 포트"에 비유합니다.

챗봇 vs 에이전트: 같은 요청, 다른 결과

요청: "내 웹사이트에 다크모드 버튼을 추가하고 서버에 반영해줘"

	챗봇	에이전트
하는 일	코드를 알려줌	파일을 직접 수정
적용	사람이 복사·붙여넣기	스스로 저장
확인	사람이 테스트	실행하고 오류를 스스로 수정
배포	사람이 업로드	명령을 실행해 반영

Agentic AI는 어디에 쓰이나?

분야	에이전트가 하는 일 (예시)
💻 **소프트웨어 개발**	기능 추가 · 버그 수정 · 테스트 실행 · 코드 리뷰
🖥️ **인프라 운영**	서버 상태 점검 · 로그 분석 · 설정 변경
📊 **데이터 분석**	파일을 읽고 정리 · 차트 생성 · 보고서 초안
🗂️ **업무 자동화**	문서 정리 · 메일 초안 · 반복 작업 처리
🔎 **리서치**	여러 자료 검색 · 비교 · 요약

ℹ️ **이 과정의 흐름**: 1~2일차 클라우드 서버 구축 → 2일차 바이브 코딩 → 3일차부터 **서버 위에서 에이전트가 직접 작업** → 이후 에이전트 확장 · 팀 프로젝트

에이전트 시대, 사람의 역할은?

역할 1
목표 정하기

무엇을 · 왜 · 어느 수준까지 만들지 **명확히** 전달

역할 2
권한 관리

AI가 **어디까지** 할 수 있는지 정하고, 위험한 작업은 **승인**

역할 3
결과 검증

화면 · 동작 · 보안을 **직접 확인**

역할 4
책임지기

배포한 결과의 **최종 책임은 사람**

🚫 **에이전트의 위험**: 잘못된 명령 실행 · 중요 파일 삭제 · 비밀 정보 노출. 그래서 **사람의 승인 (Human-in-the-loop)** 과 **백업**이 필수입니다.

오늘과 2주차의 차이

- **오늘 (1주차 2일)**
  Claude **채팅 / Desktop**으로 코드를 만들고,
  사람이 서버에 배포합니다 → 챗봇 방식으로 AI와 협업
- **2주차부터**
  에이전트 도구로 AI가 **직접** 파일을 만들고 명령을 실행하는 과정을
  눈으로 체감합니다

> 오늘 손으로 해본 과정을 기억해 두세요.
> 2주차에 이 과정을 에이전트가 대신하는 모습을 보게 됩니다.

✅ Part 1 정리

질문	답
LLM이란?	다음에 올 말을 예측해 글 · 코드를 만드는 AI의 두뇌
챗봇과 에이전트의 차이는?	챗봇은 **답을 주고**, 에이전트는 **도구로 직접 실행하고 결과를 확인**
에이전트 루프는?	목표 → 계획 → 실행 → 관찰 → (수정 반복)
에이전트 4요소는?	LLM · 도구 · 컨텍스트/메모리 · 계획
사람의 역할은?	목표 설정 · 권한 관리 · 결과 검증 · 책임

> 💬 **생각해 보기**: 어제 서버를 만들 때 했던 작업 중, 에이전트에게 맡기고 싶은 것은 무엇인가요?

Part 2

AI 3대장 비교

AI 3대장

	Anthropic	OpenAI	Google
대표 AI	**Claude**	**ChatGPT**	**Gemini**
대화형 서비스	Claude (웹·Desktop·모바일)	ChatGPT	Gemini
코딩 에이전트	Claude Code	Codex	Gemini CLI 등

AI 서비스를 쓰는 3가지 형태

형태 1

💬 채팅 서비스

웹 · 데스크톱 · 모바일 앱에서 대화
누구나
바로 사용

→

형태 2

🔌 API

내 프로그램에서 AI를 호출
개발자
가 서비스에 연결

→

형태 3

🤖 코딩 에이전트

터미널 · IDE에서 파일 · 명령 직접 처리
개발 작업
자동화

형태	Anthropic	OpenAI	Google	이 과정
채팅	Claude	ChatGPT	Gemini	**오늘 (2일차)**
API	Claude API	OpenAI API	Gemini API	이후 과정
코딩 에이전트	Claude Code	Codex	Gemini CLI 등	**3일차부터**

요금은 어떻게 매겨질까? (개념)

방식	설명	예시 상황
무료 플랜	기본 기능 · 사용량 제한	처음 써보기, 가벼운 질문
구독 (월 정액)	더 많은 사용량 · 고급 기능 · 최신 모델	매일 업무 · 학습에 사용
팀 · 기업 플랜	관리자 기능 · 좌석 단위 관리	학교 · 회사 단체 사용 (이 과정)
API 종량제	**사용한 양(토큰)** 만큼 과금	내 서비스에 AI 기능 연결

⚠️ **주의**: 구체적인 가격 · 한도는 자주 바뀝니다. 결제 전 **각 사 공식 요금 페이지**를 확인하고, API 키는 **비밀번호처럼** 관리하세요.

비교 관점

AI를 고를 때는 "어느 것이 최고인가"보다 **"무엇에 쓰는가"**가 중요합니다.

관점	확인할 질문
코딩	코드 품질, 긴 코드를 한 번에 다루는 능력
글쓰기·분석	문서 이해, 긴 자료 요약 능력
생태계	내가 쓰는 도구(문서, 메일, IDE)와의 연결
에이전트	스스로 작업을 수행하는 도구의 완성도
비용	무료 범위, 유료 요금제

3대장의 특징 (요약)

- **Claude (Anthropic)**
  코딩과 긴 문서 작업에 강점, 안전성을 중시하는 설계,
  코딩 에이전트 생태계가 활발
- **ChatGPT · Codex (OpenAI)**
  가장 널리 쓰이는 대화형 AI, 이미지·음성 등 다양한 기능,
  코딩 에이전트 Codex 제공
- **Gemini (Google)**
  Google 서비스(검색, Gmail, Docs 등)와의 연동이 강점,
  긴 입력 처리와 멀티모달

상황별로 골라 쓰기

이런 일을 할 때	확인할 점	함께 비교해 볼 질문
웹페이지 · 코드 결과물 만들기	코드 품질, 미리보기 기능	한 번에 동작하는 코드를 주는가?
긴 보고서 · 논문 요약	긴 문서 처리 능력	핵심을 빠뜨리지 않는가?
학교 · 회사 문서 도구와 연동	내가 쓰는 서비스와의 연결	Docs · 메일 · 드라이브와 연결되는가?
이미지 · 음성 활용	멀티모달 기능	사진을 이해하고 설명하는가?
비용이 중요할 때	무료 범위 · 요금제	내 사용량에 맞는가?

💡 **실무 팁**: 하나만 고집하기보다 **같은 질문을 2~3곳에 물어 비교**하면 더 좋은 답을 얻을 수 있습니다.

🧪 (활동) 같은 프롬프트, 다른 결과 비교하기

같은 요청을 여러 AI에 넣고 아래 기준으로 비교해 봅니다. (강사 시연 또는 조별 활동)

```
영남대 학생 동아리 소개 웹페이지를 index.html 한 파일로 만들어줘.
섹션: 소개, 활동, 모집 안내. 파란색 계열, 모바일 대응.
```

비교 기준	AI ①	AI ②	AI ③
요청한 섹션이 모두 있는가			
바로 실행되는 코드인가			
디자인 완성도			
설명이 이해하기 쉬운가			

이 과정에서 Claude를 쓰는 이유

1. 웹사이트 같은 **완성된 코드 결과물**을 만드는 데 강함
2. 채팅 화면에서 **결과를 바로 미리보기**하며 수정 가능
3. 2주차부터 배울 **코딩 에이전트**까지 하나의 흐름으로 연결

> 하나의 AI로 "대화 → 바이브 코딩 → 에이전트"까지 단계적으로 경험합니다.

Part 3

바이브 코딩

바이브 코딩(Vibe Coding)이란?

코드를 한 줄씩 직접 쓰는 대신,
원하는 결과를 말로 설명하고 AI가 만든 코드를 보며 다듬어 가는 개발 방식

- 2025년 초 AI 연구자 안드레 카파시(Andrej Karpathy)가 처음 쓴 표현
- "무엇을 만들지"에 집중하고, "어떻게 짤지"는 AI에게 맡김

전통적인 개발 vs 바이브 코딩

비교	전통적인 개발	바이브 코딩
시작	문법 · 언어를 먼저 배움	**만들고 싶은 결과**를 먼저 말함
코드 작성	사람이 한 줄씩 직접	**AI가 작성**, 사람은 설명 · 요청
수정	코드를 직접 찾아 고침	"이 부분을 이렇게 바꿔줘"라고 **요청**
필요한 능력	문법 · 디버깅 지식	**기획력 · 설명력 · 검증 습관**
강점	세밀한 제어, 깊은 이해	빠른 결과, 아이디어 검증

❗ **바이브 코딩에서도 기본기는 중요합니다.** HTML · CSS · JavaScript가 **어떤 역할**을 하는지 알면 AI에게 훨씬 정확하게 요청할 수 있습니다. (다음 장)

바이브 코딩의 흐름



핵심은 **한 번에 완벽하게 만들려 하지 않는 것**입니다.

작게 만들고, 보고, 고치기를 반복합니다.

웹페이지를 이루는 3가지: HTML · CSS · JavaScript

뼈대
HTML

무엇이 있는가
제목 · 문단 · 이미지 · 버튼 · 메뉴 같은 **내용과 구조**
`<h1>` `<p>` `<section>`

옷
CSS

어떻게 보이는가
색 · 글꼴 · 여백 · 배치 · **모바일 화면 대응**
`<style>` 안에 작성

동작
JavaScript

무엇을 하는가
버튼 클릭 · 다크모드 전환 · 애니메이션 같은 **기능**
`<script>` 안에 작성

내가 원하는 것	어디를 고쳐 달라고 할까?
"프로젝트 섹션을 하나 더"	HTML (내용 · 구조)
"제목을 더 크게, 파란색으로"	CSS (모양)
"버튼을 누르면 다크모드로"	JavaScript (동작)

웹페이지를 이루는 3가지: HTML · CSS · JavaScript (확대 보기)

뼈대
HTML

무엇이 있는가
제목 · 문단 · 이미지 · 버튼 · 메뉴 같은
내용과 구조

```
<h1>
<p>
<section>
```

옷
CSS

어떻게 보이는가
색 · 글꼴 · 여백 · 배치 ·
모바일 화면 대응

```
<style>
```
안에 작성

동작
JavaScript

무엇을 하는가
버튼 클릭 · 다크모드 전환 · 애니메이션 같은
기능

```
<script>
```
안에 작성

index.html 한 파일의 구조 살펴보기

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">                          <!-- 한글 깨짐 방지 -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!-- 모바일 대응 -->
  <title>홍길동의 홈페이지</title>                <!-- 브라우저 탭 제목 -->
  <style> /* CSS: 색 · 글꼴 · 배치 */ </style>
</head>
<body>
  <header> ... </header>                          <!-- HTML: 화면에 보이는 내용 -->
  <section id="projects"> ... </section>
  <script> /* JavaScript: 버튼 · 애니메이션 */ </script>
</body>
</html>
```

- 오늘 프롬프트에서 **"index.html 하나에 CSS와 JavaScript 포함"** 을 요구하는 이유: 파일 **하나만 서버에 올리면** 끝나기 때문!

index.html 한 파일의 구조 살펴보기 (확대 보기)

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">                              <!-- 한글 깨짐 방지 -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!-- 모바일 대응 -->
  <title>홍길동의 홈페이지</title>                    <!-- 브라우저 탭 제목 -->
  <style> /* CSS: 색 · 글꼴 · 배치 */ </style>
</head>
<body>
  <header> ... </header>                              <!-- HTML: 화면에 보이는 내용 -->
  <section id="projects"> ... </section>
  <script> /* JavaScript: 버튼 · 애니메이션 */ </script>
</body>
</html>
```

바이브 코딩의 장점과 함정

장점	함정
코딩 경험이 적어도 결과물을 만들 수 있음	코드를 이해하지 못한 채 배포
아이디어를 빠르게 확인	오류가 나면 원인을 모름
반복 수정이 빠름	개인정보·비밀번호를 코드에 넣는 실수

원칙: AI가 만든 코드도 **내가 책임지는 코드**입니다.
화면을 확인하고, 모르는 부분은 AI에게 "이 코드가 무슨 일을 하는지 설명해줘"라고 물어보세요.

좋은 프롬프트의 4요소

요소	예시
목표	나를 소개하는 개인 홈페이지를 만들어줘
내용	이름, 학과, 관심 분야, 프로젝트 3개
조건	HTML 파일 1개, CSS·JS는 파일 안에 포함, 한글
스타일	깔끔한 카드형 디자인, 파란색 계열, 모바일 대응

프롬프트를 더 잘 쓰는 4가지 기법

기법	방법	예시
역할 부여	AI에게 전문가 역할을 줌	"너는 웹 디자이너야. 대학생 포트폴리오를 만들어줘"
예시 제공	원하는 형태를 보여줌	"프로젝트 카드는 '제목 / 기간 / 한 줄 설명' 형태로"
단계 나누기	큰 작업을 작게 쪼갬	"먼저 레이아웃만, 다음에 색상, 마지막에 애니메이션"
제약 조건	꼭 지킬 조건 명시	"외부 이미지 사용 금지, 파일 1개, 한글 UTF-8"

💡 **막막할 때 쓰는 마법의 문장**: "내가 원하는 웹사이트를 만들기 위해 **필요한 정보를 나에게 질문해줘.**" → AI가 질문하고, 내가 답하면서 기획이 완성됩니다.

나쁜 프롬프트 vs 좋은 프롬프트

나쁜 예

> 홈페이지 만들어줘

좋은 예

> 영남대 컴퓨터공학과 2학년 학생의 자기소개 홈페이지를 만들어줘.
> - 섹션: 소개, 관심 분야, 프로젝트(3개), 연락처
> - index.html 파일 하나에 CSS와 JavaScript를 모두 포함
> - 한글이 깨지지 않도록 UTF-8 설정
> - 파란색 계열의 깔끔한 디자인, 모바일 화면에서도 보기 좋게

수정 요청은 구체적으로

막연한 요청	구체적인 요청
더 예쁘게 해줘	제목 글씨를 더 크게, 섹션 사이 여백을 넓게 해줘
뭔가 추가해줘	오른쪽 위에 다크모드 전환 버튼을 추가해줘
이상해	모바일에서 프로젝트 카드가 겹쳐 보여. 한 줄에 하나씩 나오게 해줘

> 한 번에 하나씩 요청하면 결과를 확인하기 쉽습니다.

반복 개선 대화 예시 (v1 → v3)

🙋 **1차 요청** 영남대 컴퓨터공학과 학생 자기소개 홈페이지 (4요소 프롬프트) → **v1 완성**: 소개 · 관심 분야 · 프로젝트 · 연락처

▼ 미리보기 확인: "제목이 작고 섹션이 붙어 있다"

🙋 **2차 요청** "제목 글씨를 더 크게, 섹션 사이 여백을 넓게 해줘" → **v2**: 읽기 편해짐

▼ 미리보기 확인: "밤에 보기엔 눈이 부시다"

🙋 **3차 요청** "오른쪽 위에 다크모드 전환 버튼을 추가해줘" → **v3**: 기능 추가 완료 ✅

- 매 단계 **하나만** 바꾸고 **바로 확인** → 문제가 생겨도 원인을 쉽게 찾을 수 있습니다.
- 마음에 드는 버전은 **코드를 따로 저장**해 두면 언제든 돌아갈 수 있습니다.

오류가 났을 때 쓰는 프롬프트 패턴

상황	이렇게 요청하세요
화면이 깨짐	"모바일에서 카드가 겹쳐 보여. **원인을 설명하고** 고친 **전체 코드**를 줘"
버튼이 동작 안 함	"다크모드 버튼을 눌러도 아무 변화가 없어. **무엇을 확인**하면 될까?"
이전이 더 좋았음	"방금 수정은 취소하고 **바로 전 버전**으로 돌아가줘"
오류 메시지가 있음	오류 문구를 **그대로 복사해** 붙여넣고 "이 오류의 원인과 해결 방법을 알려줘"
너무 많이 바뀜	"**다른 부분은 그대로 두고** 헤더 색상만 바꿔줘"

❗ **상황을 정확히 전달**하는 것이 핵심입니다: **무엇을 했고 → 무엇을 기대했고 → 실제로 어떻게 됐는지**

AI가 만든 코드를 이해하는 질문들

> 방금 만든 코드를 초보자도 이해할 수 있게 부분별로 설명해줘.

> <style> 안에서 색상을 바꾸려면 어느 줄을 고치면 돼?

> 다크모드 버튼이 동작하는 원리를 3줄로 설명해줘.

> 이 코드에 개인정보나 보안상 문제가 될 만한 부분이 있는지 확인해줘.

> 💡 이해한 만큼 **수정 요청이 정확해지고**, 문제가 생겼을 때 **스스로 해결**할 수 있습니다.

✅ 배포 전 바이브 코딩 체크리스트

확인 항목	확인 방법
요청한 섹션이 모두 있다	미리보기 화면 확인
한글이 깨지지 않는다	`<meta charset="UTF-8">` 포함 여부
모바일에서도 보기 좋다	브라우저 창을 좁게 줄여 보기
버튼 · 기능이 동작한다	직접 클릭해 보기
개인정보가 없다	전화번호 · 주소 · 학번 등 삭제
외부 파일에 의존하지 않는다	파일 **하나**로 동작하는지 확인
코드 **전체**를 복사했다	`<!DOCTYPE html>` 로 시작해 `</html>` 로 끝나는지

Part 4

네이버클라우드의 웹서버로 배포

배포란?

내 노트북에서 만든 파일을 서버로 옮겨, 누구나 인터넷으로 볼 수 있게 하는 것

[Claude가 만든 index.html]

복사 →

[내 노트북]

SSH 원격 접속 →

[Ubuntu Server]

`/var/www/html/index.html`

→

http://<공인 IP>

로 누구나 접속

브라우저와 웹서버는 어떻게 대화할까?

① 사용자

주소 입력

`http://<공인 IP>`

HTTP 요청 →

80번 포트

② ACG

80번 허용?

허용된 요청만 통과

→

③ 웹서버

Nginx

`/var/www/html/`

에서

`index.html`

찾기

HTTP 응답 ←

HTML 전달

④ 브라우저

화면 그리기

HTML · CSS · JS를
해석해 표시

용어	의미
클라이언트	요청하는 쪽 (내 브라우저)
서버	응답하는 쪽 (네이버클라우드의 Ubuntu 서버)
HTTP	웹 페이지를 주고받는 약속(프로토콜), 기본 **80번 포트**

> 💡 그래서 배포는 **서버의** `index.html` **파일만 바꾸면** 브라우저 화면이 바뀝니다.

브라우저와 웹서버는 어떻게 대화할까? (확대 보기)

① 사용자

주소 입력

`http://<공인 IP>`

HTTP 요청 →

80번 포트

② ACG

80번 허용?

허용된 요청만 통과

→

③ 웹서버

Nginx

`/var/www/html/` 에서 `index.html` 찾기

← HTTP 응답

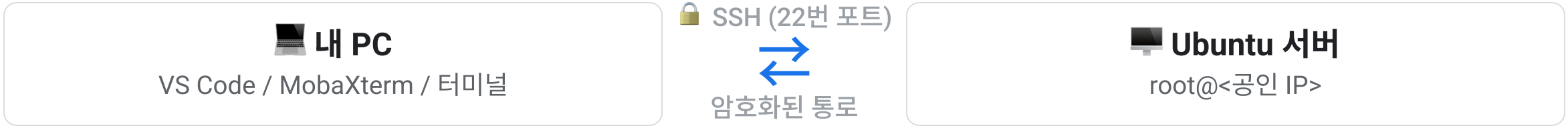
HTML 전달

④ 브라우저

화면 그리기

HTML · CSS · JS를 해석해 표시

SSH — 서버를 원격으로 조작하는 통로



- **SSH (Secure Shell)**: 내 PC에서 멀리 있는 서버에 **안전하게(암호화)** 접속해 명령을 내리고 파일을 주고받는 방법
- 접속 정보 3가지: **주소**(공인 IP) · **계정**(root) · **비밀번호**(root 비밀번호)
- 파일을 옮기는 **SFTP**도 SSH 통로를 이용합니다.

🚫 SSH 비밀번호가 있으면 서버의 **모든 것**을 바꿀 수 있습니다. 절대 공유하거나 AI 채팅창에 입력하지 마세요.

웹서버의 문서 루트

- 웹서버는 정해진 폴더의 파일을 브라우저에 보여줍니다
- Ubuntu 웹서버의 기본 폴더(문서 루트): **`/var/www/html`**
- 이 폴더의 `index.html`이 첫 화면이 됩니다

> 오늘 할 일 = 이 폴더의 `index.html`을 **내가 만든 파일로 교체**

원격 접속 방법: 목표는 하나, 길은 여러 개

방법	특징	추천 대상
VS Code Remote SSH	서버 파일을 내 PC처럼 편집	기본 방법
MobaXterm	터미널 + 파일 끌어다 놓기 업로드	VS Code 연결이 어려운 경우 (Windows)
터미널 `ssh` · `scp`	설치 없이 기본 제공 · 명령으로 업로드	VS Code 연결이 어려운 경우 (Mac · Linux)

> 어떤 방법을 쓰든 `/var/www/html/index.html`**이 바뀌면 성공**입니다.

접속 도구는 어떻게 동작할까?

A VS Code Remote SSH

- 접속하면 서버에 **VS Code Server**가 자동 설치됨
- 서버의 파일을 **내 PC 편집기처럼** 열고 수정 · 저장
- 저장(`Ctrl+S` · Mac `⌘+S`)하는 순간 **서버 파일이 바로 바뀜**
- 터미널도 VS Code 안에서 사용

B MobaXterm

- 터미널 + 왼쪽 **SFTP 파일 탐색기**
- 내 PC의 `index.html`을 서버 폴더로 **끌어다 놓기**
- 같은 이름 파일은 **덮어쓰기**
- Windows 전용, 설치가 간단
- **C Mac · Linux**: 기본 터미널에서 `ssh` 접속 · `scp` 로 업로드

비교	VS Code Remote SSH	MobaXterm	터미널 (Mac · Linux)
오늘 배포 방법	서버의 `index.html`을 열어 **코드 붙여넣기 → 저장**	PC에 저장한 `index.html`을 **드래그 앤 드롭**	PC에 저장한 `index.html`을 **`scp` 로 업로드**
이럴 때 추천	기본 방법 (모든 OS)	VS Code가 안 될 때 (Windows)	VS Code가 안 될 때 (Mac)

접속 도구는 어떻게 동작할까? — 비교표 (확대 보기)

비교	VS Code Remote SSH	MobaXterm	터미널 (Mac · Linux)
오늘 배포 방법	서버의 `index.html`을 열어 **코드 붙여넣기 → 저장**	PC에 저장한 `index.html`을 **드래그 앤 드롭**	PC에 저장한 `index.html`을 **`scp`로 업로드**
이럴 때 추천	기본 방법 (모든 OS)	VS Code가 안 될 때 (Windows)	VS Code가 안 될 때 (Mac)

배포 전 확인할 3가지

1. 서버 **공인 IP**와 **root 비밀번호**를 알고 있는가?
2. **ACG**에서 **22번(SSH)**, **80번(HTTP)** 포트가 열려 있는가?
3. 웹서버가 **실행 중**인가? (어제 페이지가 브라우저에서 보이는가?)

> 셋 중 하나라도 안 되면 실습 문서의 "자주 막히는 지점"을 확인하세요.

백업 → 교체 → 확인: 안전한 배포 3단계

1단계

백업

```
cp index.html
index_day1.html
```

어제 파일 보관

→

2단계

교체

VS Code 붙여넣기 · 저장
또는 MobaXterm 드래그 ·

```
scp
```

→

3단계

확인

```
head -n 10 index.html
```

브라우저

```
Ctrl + F5
```

(Mac

```
⌘+Shift+R
```

)

- **백업**: 문제가 생기면 `cp index_day1.html index.html` 한 줄로 **원래대로 복구**
- **확인**: 서버 파일 내용(터미널)과 실제 화면(브라우저)을 **둘 다** 확인
- 예전 화면이 보이면 브라우저 **캐시** 때문일 수 있음 → 강력 새로고침

🔧 "안 보여요!" 원인 찾기 지도

증상	의심할 곳	확인 방법
사이트가 아예 안 열림	**ACG 80번** · 웹서버 정지	콘솔 ACG 규칙 · `systemctl status nginx`
원격 접속이 시간 초과	**ACG 22번** · 공인 IP 오타	콘솔 ACG 규칙 · 기록 카드 IP 비교
예전 페이지가 그대로	저장 안 함 · **캐시**	`Ctrl+S` 확인 · `Ctrl+F5` (Mac `⌘+Shift+R`)
한글이 깨짐	**인코딩**	`<meta charset="UTF-8">` · UTF-8 저장
화면이 하얗게 비어 있음	코드 **일부만** 복사	`head` · `tail` 로 파일 앞뒤 확인

> 💡 문제를 **"네트워크(ACG) → 서버(웹서버) → 파일(내용) → 브라우저(캐시)"** 순서로 좁혀 가면 빠르게 찾을 수 있습니다.

보안 기본 수칙

- 비밀번호, 인증키(.pem) 파일은 **절대 다른 사람과 공유하지 않기**
- 웹페이지에 **전화번호·주소 같은 개인정보 넣지 않기**
- AI 채팅창에 **서버 비밀번호를 입력하지 않기**
- 실습이 끝나면 강사 안내에 따라 **서버 정지 / 반납**

오늘 실습의 목표

1. Claude로 **나만의 웹사이트** 만들기 (바이브 코딩)
2. 프롬프트로 **2회 이상 고도화** 하기
3. **VS Code Remote SSH** 또는 **MobaXterm**(Windows) · **터미널**(Mac)로 서버 접속
4. `/var/www/html/index.html` 교체
5. `http://<공인 IP>`에서 **내 웹사이트 확인**

2주차 예고

- 오늘: **내가** 코드를 복사하고, **내가** 서버에 올렸습니다
- 2주차: **AI 에이전트가** 파일을 만들고, 명령을 실행하고, 오류를 고치는 모습을 직접 체감합니다

> "오늘 손으로 한 작업 중 어떤 부분을 AI에게 맡기고 싶었나요?"
> 이 질문을 가지고 2주차를 시작합니다.

📌 오늘의 핵심 정리

Part	핵심 한 줄
Part 1 · Agentic AI	에이전트는 **목표를 받아 계획 · 실행 · 관찰 · 수정**을 스스로 반복하는 AI
Part 2 · AI 3대장	"어느 것이 최고"보다 **"무엇에 쓰는가"** 로 고른다
Part 3 · 바이브 코딩	**4요소 프롬프트**로 요청하고, **작게 만들고 · 보고 · 고치기**를 반복한다
Part 4 · 배포	**백업 → 교체 → 확인**, 서버의 `index.html` 이 바뀌면 배포 완료

📖 오늘의 용어 사전

용어	뜻	용어	뜻
LLM	대규모 언어 모델, AI의 두뇌	**바이브 코딩**	말로 설명하고 AI 코드를 다듬는 개발
프롬프트	AI에게 주는 요청 · 지시문	**환각**	AI가 그럴듯한 오답을 말하는 현상
컨텍스트	AI가 참고하는 대화 · 자료 전체	**에이전트**	도구로 스스로 실행하는 AI
MCP	AI와 도구를 연결하는 표준	**API**	프로그램끼리 기능을 주고받는 창구
HTTP	웹 페이지를 주고받는 약속 (80번)	**SSH**	암호화된 원격 접속 (22번)
문서 루트	웹 파일 폴더 `/var/www/html`	**배포**	만든 파일을 서버에 올려 공개

✅ 셀프 체크 퀴즈

1. 챗봇과 에이전트의 가장 큰 차이는 무엇인가요?
2. 에이전트의 4가지 구성 요소는?
3. 좋은 프롬프트의 4요소는?
4. "다크모드 버튼 추가"는 HTML · CSS · JavaScript 중 주로 어디를 바꾸는 요청일까요?
5. 배포 후 예전 화면이 계속 보일 때 가장 먼저 해볼 것은?

> **정답**: ① 챗봇은 답을 주고, 에이전트는 도구로 직접 실행 · 확인 ② LLM · 도구 · 컨텍스트/메모리 · 계획 ③ 목표 · 내용 · 조건 · 스타일 ④ JavaScript (동작) + CSS (모양) ⑤ `Ctrl + F5` (Mac `⌘+Shift+R`) 강력 새로고침 (저장 여부도 확인)

Q & A

이제 실습을 시작합니다